

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа по
Вычислительной математике №1

Работу выполнил:

Гуменник П. О.

Группа:

P3233

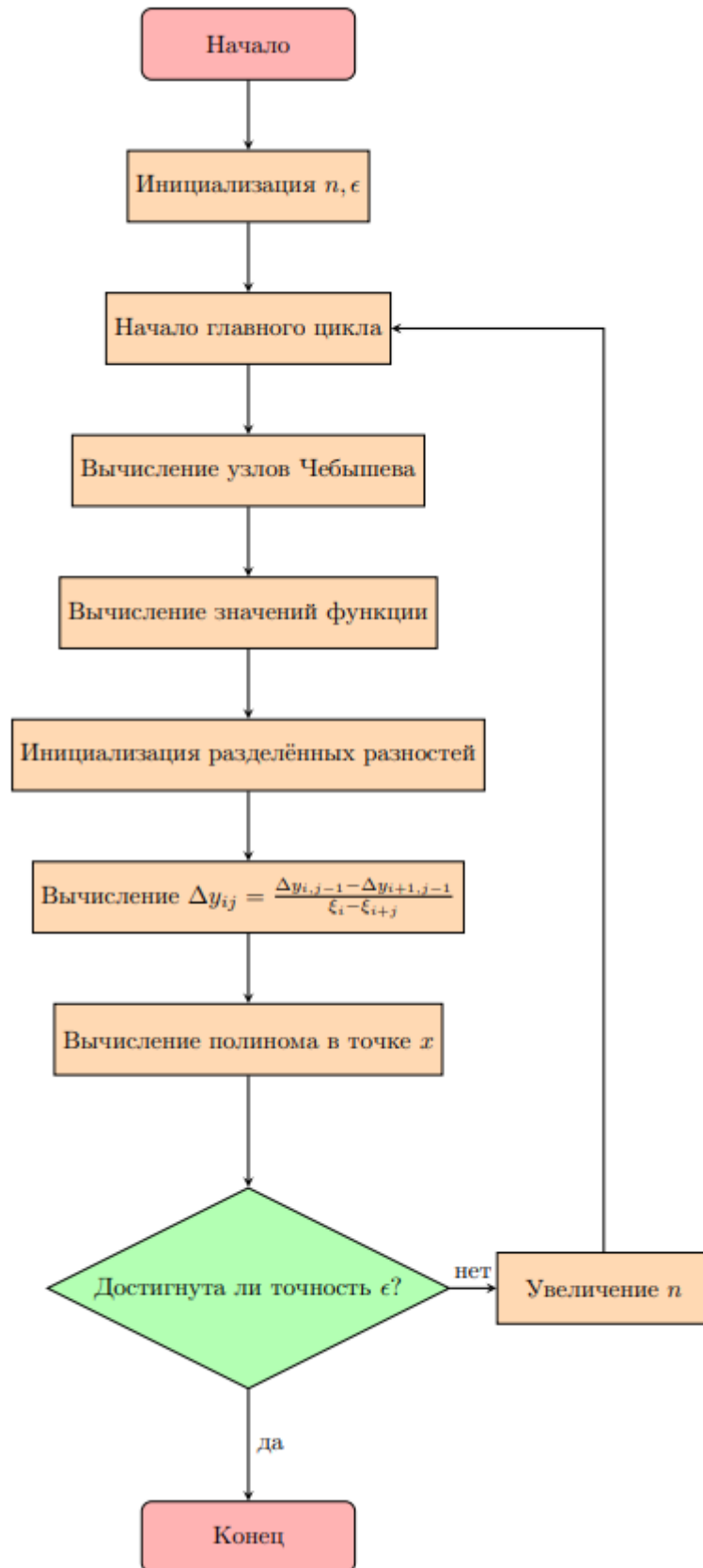
Санкт-Петербург,

2024

Задание

Дан набор точек, по которым необходимо построить интерполяционный полином Ньютона с использованием полиномов Чебышёва для заданного интервала. Также задана координата x , для которой необходимо найти значение интерполяционного полинома. Формат входных данных: $f \ a \ b \ x$ где f - номер интерполируемой функции, a и b - границы интерполируемого интервала и x - значение аргумента для интерполяционного полинома. Степень полинома Чебышева (количество узлов интерполяции) следует увеличивать до тех пор, пока модуль разницы между значениями интерполирующей функции в искомой точке x не будет меньше чем 0.01. Формат выходных значений: вещественное число, являющееся значением интерполяционного полинома в точке x .

Блок-схема



Код программы:

```
#!/bin/python3

import math

class FunctionSet:

    def weierstrass_function(x: float):
        f_x = 0
        n = 5
        b = 0.5
        a = 13
        for i in range(n):
            f_x += pow(b, i) * math.cos(pow(a, i) * math.pi * x)

        return f_x

    # How to use this function:
    # func = FunctionSet.get_function(4)
    # func(0.01)
    def get_function(n: int):
        if n == 1:
            return FunctionSet.weierstrass_function
        elif n == 2:
            return math.gamma
        elif n == 3:
            return math.exp
        else:
            raise NotImplementedError(f"Function {n} not defined.")

def chebyshev_nodes(a, b, n):
    return [0.5 * ((a + b) + (b - a) * math.cos((2 * i + 1) * math.pi / (2 * n))) for i in range(n)]

def divided_diff(x, y):
    n = len(y)
    coef = [0] * n
    coef[0] = y[0]

    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            y[i] = (y[i] - y[i - 1]) / (x[i] - x[i - j])
            coef[j] = y[j]
    return coef

def newton_poly(coef, x_data, x):
    n = len(coef) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n - k] + (x - x_data[n - k]) * p
    return p

def interpolate_by_newton(f, a, b, x):
    n = 1 # Starting number of nodes
    difference = float('inf') # Initialize difference
    p_prev = None
    func = FunctionSet.get_function(f)
```

```

# Keep increasing the number of nodes until difference < 0.01
while difference >= 0.01:
    # Generate Chebyshev nodes and corresponding y values
    x_data = chebyshev_nodes(a, b, n)
    y_data = [func(xi) for xi in x_data]

    # Calculate the divided differences (Newton coefficients)
    coefficients = divided_diff(x_data, y_data)

    # Evaluate the Newton polynomial
    p = newton_poly(coefficients, x_data, x)

    # Update the difference if p_prev is defined
    if p_prev is not None:
        difference = abs(p - p_prev)

    # Update p_prev for the next iteration
    p_prev = p

    # Increase the number of nodes
    n += 1
    if n > 150:
        raise Exception("Converging error")

return p

if __name__ == '__main__':
    print("""
    f(x):
    1 - Weierstrass function
    2 - Gamma function
    3 - Exp
    4 - Sin
    """)
    f = int(input('> input f(x): ').strip())
    a = float(input('> a: ').strip())
    b = float(input('> b: ').strip())
    x = float(input('> x: ').strip())

    try:
        result = interpolate_by_newton(f, a, b, x)
        print("Result: ", str(result) + '\n')

        f = FunctionSet.get_function(f)
        print("f(x) = ", f(x))
    except Exception as e:
        print(e.args)

```

Результаты работы программы:

1)

f(x):

1 - Weierstrass function

2 - Gamma function

3 - Exp

4 - Sin

> input f(x): 2

> a: 2

> b: 6

> x: 5

Result: 24.007697264576734

$f(x) = 24.0$

2)

> input f(x): 3

> a: 2

> b: 5

> x: 9

Result: 8102.657169758276

$f(x) = 8103.083927575384$

3)

> input f(x): 1

> a: 0

> b: 2

> x: 1.6

Result: 0.053900065387249674

$f(x) = -0.10005081636769754$

4)

> input f(x): 2

> a: -2

> b: -1

> x: -1.5

('Converging error',)

5)

> input f(x): 4

> a: 0

> b: 3.1415

> x: 1.5708

Result: 0.9993899080946698

$f(x) = 0.9999999999932537$

Вывод:

Комментарии к тестовым примерам:

1) Результат интерполяции гамма-функции достаточно близок к фактическому значению функции. Это говорит о том, что в данном случае полиномиальная аппроксимация достаточно эффективна на этом интервале.

2) Несмотря на то, что значение x находится за пределами заданного интервала, интерполированный результат очень близок к фактическому значению функции. Это несколько неожиданно, поскольку экстраполяция (интерполяция за пределами заданного интервала) обычно дает менее надежные результаты по сравнению с интерполяцией внутри интервала.

3) Это показывает существенное расхождение между интерполяцией и фактическими значениями функции. Функция Вейерштрасса известна своей сложностью и непрерывным, но нигде не дифференцируемым свойством, что может сделать ее особенно сложной для полиномиальной интерполяции.

4) Это указывает на то, что интерполяция не сходилась в ожидаемом количестве узлов или итераций (более 150 узлов). Это может быть связано с природой гамма-функции, которая имеет особенности (полюсы) и быстро меняющееся поведение в отрицательной области, особенно вблизи неположительных целых чисел. Интерполяция в таких регионах может быть проблематичной.

5) В этом случае результат интерполяции чрезвычайно близок к фактическому значению синусоидальной функции при $\pi/2$. Такая эффективность обусловлена несколькими факторами:

Хорошая функция: синусоидальная функция является гладкой и периодической, что делает ее отличным кандидатом для полиномиальной интерполяции.

Симметрия и границы: интервал интерполяции охватывает одну полную волну от 0 до π , который помогает точно отразить поведение функции.

Местоположение x : выбранный x расположен в центре интервала, где полиномиальная интерполяция обычно обеспечивает наилучшие приближения.

Сравнение

Преимущества формулы Ньютона по сравнению с другими методами:

- Легче добавлять новые точки без перестроения всего полинома.
- Требуется меньшее число точек для приближения таких функций как парабола.

Недостатки:

- Шум существенно влияет на весь полином.

Использование узлов Чебышева уменьшает проблему Рунге по сравнению с равноотстоящими узлами, что делает этот подход более эффективным для функций с проблемной интерполяцией, особенно на больших интервалах.

Применимость

1. Функция Вейерштрасса

Метод Ньютона, особенно с узлами Чебышева, может пытаться сгладить осцилляции функции Вейерштрасса, но может не достичь хорошей точности из-за высокой осцилляторности. Это может привести к нестабильности интерполяции.

2. Гамма-функция

В положительном диапазоне и на отрезках без особенностей метод Ньютона с узлами Чебышева может быть эффективен для интерполяции гамма-функции, обеспечивая хорошую точность и избегая проблемы Рунге.

Алгоритмическая сложность — $O(n^2)$

1. Ошибка интерполяции:

Основная ошибка в методе Ньютона, связанная с разницей между истинной функцией $f(x)$ и интерполяционным полиномом $P_n(x)$, для полинома степени n , если функция достаточно гладкая (имеет непрерывные производные до порядка $n+1$), тогда максимальная теоретическая ошибка интерполяции на интервале может быть выражена через производную порядка $n+1$ от функции:

$$E(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

где ξ находится где-то в интервале интерполяции, а x_i - это узлы интерполяции. Эта ошибка становится больше, когда мы движемся к краям интервала интерполяции, что проявляется в явлении Рунге.

2. Численная ошибка:

Помимо теоретической ошибки интерполяции, численные ошибки могут возникать из-за арифметики с плавающей запятой при вычислении разделенных разностей и значений полинома. Эти ошибки обычно небольшие при низких степенях полинома, но могут увеличиваться с увеличением степени полинома из-за увеличения числа операций.

3. Ошибка за счет выбора узлов:

Использование узлов Чебышева минимизирует, но не полностью исключает ошибку интерполяции, особенно в случаях с высокоосциллирующими функциями или функциями, которые плохо аппроксимируются полиномами. Эти узлы помогают избежать явления Рунге, но все же не гарантируют идеальную точность для всех функций.

4. Ошибка конвергенции:

Метод предполагает, что при увеличении числа узлов интерполяции (и степени полинома) ошибка между последовательными аппроксимациями будет уменьшаться. Однако, для некоторых функций и выбора интервалов, может не существовать гарантии, что разность между последовательными аппроксимациями будет уменьшаться до приемлемого уровня, особенно если функция ведет себя нестандартно или если интервал слишком велик.